Eager Loading vs Lazy Loading

EF Core • Concise Lesson Summary

Core idea: Both techniques load related data through navigation properties. The key difference is **when** EF Core loads that data.

1. Eager Loading

Definition: Related data is loaded immediately with the initial query, using **Include()**.

```
var students = context.Students
    .Include(s => s.Enrollments)
    .ToList();
```

What happens? ToList() executes the query. EF Core loads Students and their Enrollments together. For a collection navigation, SQL often uses a LEFT JOIN; the exact SQL depends on the query and provider.

Mental model: "I know I need the related data, so load it now."

2. Lazy Loading

Definition: Related data is not loaded initially. EF Core loads it automatically when the navigation property is accessed.

```
var students = context.Students.ToList();

foreach (var student in students)
{
    Console.WriteLine(student.Enrollments.Count);
}
```

What happens? The first query loads Students only. Accessing student.Enrollments can trigger another SQL query for that student's related data.

Mental model: First query → Students only. Later access → EF Core loads related data.

3. How to Enable Lazy Loading

Lazy loading with proxies generally requires the **Microsoft.EntityFrameworkCore.Proxies** package, proxy-compatible entity classes, and **virtual** navigation properties.

```
// Navigation property
public virtual ICollection<Enrollment> Enrollments { get; set; }

// DbContext configuration
options
    .UseLazyLoadingProxies()
    .UseSqlServer(connectionString);
```

Why virtual? EF Core creates a proxy object that can intercept navigation-property access and trigger the related-data query. The DbContext must still be available.

4. The N+1 Problem

If the initial query loads 100 students and the loop accesses Enrollments for every student, lazy loading may execute **1 query for Students + 100 queries for Enrollments = 101 queries**.

1 Students query	100 Related-data queries	101 Total queries

Why it matters: The number of database round trips grows with the number of parent entities. This pattern is called the **N+1 Problem**.

5. When Should You Use Each?

Eager Loading	Lazy Loading
• APIs • Reports and dashboards • Predictable query behavior • Performance-sensitive paths	• Small internal apps • Prototypes • Quick admin tools • Small datasets where deferred loading is acceptable

Key Takeaways

• **Eager Loading:** use Include() to load related data as part of the initial query.

• **Lazy Loading:** related data loads when its navigation property is accessed.

• **N+1:** lazy loading can generate one query for the parent list plus one query per parent.

• For backend APIs, prefer predictable data access and load only the data the use case needs.

• Eager Loading is not automatically faster; large joins or loading too much data can also hurt performance.

Choose the loading strategy based on the data your operation needs and the SQL it generates.